

ESTRUTURA DE DADOS I

Professor: Diego Ricardo Krohl

IMPLEMENTAÇÃO DE TABELA HASH COM ENCADEAMENTO E LISTA DUPLA

João Vitor Vitososki

Videira – SC
Novembro de 2025

1 Introdução

Este relatório documenta a implementação de uma tabela hash com tratamento de colisões por encadeamento separado, utilizando listas duplamente encadeadas. O objetivo principal foi criar uma estrutura de dados capaz de armazenar e manipular um conjunto de 100.788 nomes brasileiros, avaliando se a performance da estrutura implementada esta ok.

A eficiência de uma tabela hash depende diretamente da qualidade da função hash utilizada e do método de tratamento de colisões implementado.

2 Metodologia

2.1 Arquitetura do Sistema

A implementação foi organizada em 7 módulos distintos para facilitar manutenção e compreensão:

- **hash_table.h**: Declarações de tipos e protótipos de funções
- **hash_functions.c**: Implementação da função hash DJB2
- **hash_table_ops.c**: Operações básicas (inserção, busca, remoção)
- **sorting.c**: Algoritmo QuickSort para ordenação e ordenação de valores dentro de chave específica
- **statistics.c**: Análise estatística e geração de histograma
- **file_utils.c**: Carregamento do dataset
- **main.c**: Interface de usuário e menu interativo

A compilação foi automatizada através de script `.bat`, pois fiz no windows e não suporta o Makefiles para compilar os arquivos.

2.2 Escolha da Função Hash: DJB2

Fiz algumas pesquisas e optei pela função **DJB2** (Daniel J. Bernstein) devido às seguintes características:

Fórmula: $hash = hash \times 33 + c$ para cada caractere c

Justificativa da escolha:

1. **Simplicidade computacional:** Operação de deslocamento de bits eficiente ($hash \ll 5 + hash$ equivale a $hash \times 33$)
2. **Efeito avalanche superior:** Pequenas mudanças na entrada produzem grandes variações no hash
3. **Número mágico 33:** Valor primo que comprovadamente minimiza colisões em strings (e também pq o professor deu a dica kkkk)
4. **Processamento completo:** Considera todos os caracteres da string, não apenas prefixos

5. **Performance:** Complexidade $O(n)$ onde n = comprimento da string, com constante baixa

A função DJB2 foi preferida em relação ao exemplo fornecido (baseado apenas no primeiro caractere) por proporcionar distribuição significativamente mais uniforme, especialmente para nomes com prefixos comuns na língua portuguesa.

Listing 1: Implementação da função hash DJB2

```
1 int hashFunction(const char *name, int tableSize) {
2     unsigned long hash = 5381;
3     int c;
4
5     while ((c = *name++)) {
6         hash = ((hash << 5) + hash) + c; // hash * 33 + c
7     }
8
9     return hash % tableSize;
10 }
```

2.3 Estruturas de Dados

Foram implementadas duas estruturas principais:

Listing 2: Estruturas de dados utilizadas

```
1 typedef struct Node {
2     char name[MAX_NAME];
3     struct Node *next;
4     struct Node *prev;
5 } Node;
6
7 typedef struct {
8     Node **table;
9     int *counts;
10    int size;
11 } HashTable;
```

A **lista duplamente encadeada** permite remoção em tempo $O(1)$ após localização do elemento, enquanto a **tabela hash** com array de contadores facilita análise estatística sem necessidade de percorrer as listas.

Parâmetros da implementação:

- Tamanho da tabela (m): 53 (número primo próximo de 50)
- Dataset: 100.788 nomes brasileiros
- Fator de carga teórico: $\alpha = \frac{100.788}{53} \approx 1.901,66$

3 Tratamento de Colisões

3.1 Método Implementado: Encadeamento Separado

O encadeamento separado foi escolhido pelas seguintes vantagens:

- **Simplicidade:** Implementação direta e compreensível
- **Flexibilidade:** Sem limitação do número de elementos
- **Performance estável:** Degradação gradual mesmo com alto fator de carga
- **Remoção eficiente:** $O(1)$ com lista duplamente encadeada

3.2 Quando é Necessário?

Com 100.788 elementos distribuídos em apenas 53 chaves, colisões são matematicamente inevitáveis pelo **Princípio das Casas dos Pombos**. O fator de carga $\alpha \approx 1.902$ indica que cada chave armazena, em média, aproximadamente 1.902 elementos, tornando o tratamento de colisões essencial para o funcionamento da estrutura.

4 Operações Implementadas

4.1 Complexidade Algorítmica

A Tabela 1 apresenta a complexidade das principais operações:

Tabela 1: Complexidade algorítmica das operações

Operação	Média	Pior Caso
Inserção	$O(1)$	$O(n)$
Busca	$O(1 + \alpha)$	$O(n)$
Remoção	$O(1 + \alpha)$	$O(n)$
Ordenação	$O(k \log k)$	$O(k^2)$

Onde α = fator de carga e k = elementos na chave sendo ordenada.

4.2 Algoritmo de Ordenação: QuickSort

Implementou-se o QuickSort conforme especificação, com:

- Particionamento usando último elemento como pivô
- Ordenação lexicográfica via `strcmp()`
- Complexidade média $O(n \log n)$
- Aplicado individualmente a cada chave quando solicitado

5 Análise de Resultados

5.1 Dados Obtidos

Com base nas estatísticas geradas pelo sistema, foram obtidos os seguintes resultados:

Tabela 2: Estatísticas da tabela hash

Métrica	Valor
Total de elementos	100.788
Tamanho da tabela	53 chaves
Chaves vazias	0 (0,00%)
Maior cadeia	1.994 elementos
Menor cadeia	1.807 elementos
Média por chave	1.901,66 elementos
Desvio padrão	40,66 elementos
Fator de carga	1.901,6604

5.2 Avaliação da Uniformidade

Coefficiente de Variação: 2,14%

Este valor indica distribuição **EXCELENTE**, significativamente abaixo do limite de 10% estabelecido para distribuições uniformes. O coeficiente de variação mede a dispersão relativa dos dados: quanto menor, mais uniforme a distribuição.

Razão Máximo/Média: 1,05

A razão entre a maior cadeia (1.994) e a média (1.901,66) resulta em 1,05, valor muito próximo ao ideal de 1,0. Isso demonstra que não há concentração excessiva em nenhuma chave específica.

Amplitude: $1.994 - 1.807 = 187$ elementos

A diferença entre a maior e menor cadeia representa apenas 9,8% da média, confirmando a homogeneidade da distribuição.

5.3 Teste da Hipótese do Hashing Uniforme

Os resultados obtidos confirmam que a implementação da tabela hash e a hipótese de hashing foi alcançada pelos seguintes critérios:

Coefficiente de variação menor que 10%: 2,14% (EXCELENTE)

Razão máximo/média menor que 1,5: 1,05 (IDEAL)

Nenhuma chave vazia: 0% de desperdício

Desvio padrão controlado: 40,66 (2,14% da média)

5.4 Análise do Histograma

O histograma gerado apresenta distribuição visual praticamente uniforme, com todas as 53 chaves apresentando valores entre 1.807 e 1.994 elementos. A ausência de picos ou vales acentuados confirma:

- **Não há chaves sobrecarregadas:** Evita degradação de performance
- **Não há chaves vazias:** Aproveitamento ótimo da memória
- **Distribuição homogênea:** Garante tempo de busca consistente

Esta uniformidade é particularmente notável considerando as características linguísticas do português brasileiro, onde certos prefixos nominais são mais comuns (ex: “Maria”, “João”, “Ana”).

6 Análise de Performance

6.1 Tempo de Operações

Com fator de carga $\alpha \approx 1,902$ e distribuição uniforme:

- **Busca:** Tempo esperado = $O(1 + \alpha) \approx O(2,9) \approx O(1)$ — praticamente constante
- **Inserção:** $O(1)$ — inserção no início da lista
- **Remoção:** $O(1)$ após busca — benefício da lista duplamente encadeada

6.2 Uso de Memória

- **Tabela principal:** $53 \times 8 = 424$ bytes
- **Array de contadores:** $53 \times 4 = 212$ bytes
- **Nós:** $100.788 \times (100 + 16) \approx 11,1$ MB
- **Total aproximado:** $\sim 11,1$ MB

O overhead da estrutura representa menos de 1% do total, demonstrando eficiência na gestão de memória.